

Walmart

May 17, 2025

About Walmart

Walmart, founded in 1962 by Sam Walton, is a retail giant and one of the world's largest and most influential companies. Headquartered in Bentonville, Arkansas, this American multinational corporation has established itself as a global powerhouse in the retail industry. Walmart operates a vast network of hypermarkets, discount department stores, and grocery stores under various brand names across the United States and in numerous countries around the world.

Known for its "Everyday Low Prices" strategy, Walmart has redefined the retail landscape with its commitment to offering a wide range of products at affordable prices. With its extensive supply chain and efficient distribution systems, the company has played a pivotal role in shaping consumer expectations and shopping habits. Beyond retail, Walmart has also ventured into e-commerce, technology innovation, and sustainability initiatives, further solidifying its position as a key player in the modern retail ecosystem.

About Data

The company collected the transactional data of customers who purchased products from the Walmart Stores during Black Friday. It has information of about 0.5 Million transactions during Black Friday throughout various years.

```
[3]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import statistics
import warnings
warnings.filterwarnings("ignore")
```

Loading Dataset

```
[19]: walmart = pd.read_csv('walmart_data.csv')
walmart
```

```
[19]:
```

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	\
0	1000001	P00069042	F	0-17	10	A	
1	1000001	P00248942	F	0-17	10	A	
2	1000001	P00087842	F	0-17	10	A	
3	1000001	P00085442	F	0-17	10	A	
4	1000002	P00285442	M	55+	16	C	

...	...	...	...	...	...	...	...
550063	1006033	P00372445	M	51-55	13	B	
550064	1006035	P00375436	F	26-35	1	C	
550065	1006036	P00375436	F	26-35	15	B	
550066	1006038	P00375436	F	55+	1	C	
550067	1006039	P00371644	F	46-50	0	B	

	Stay_In_Current_City_Years	Marital_Status	Product_Category	Purchase
0	2	0	3	8370
1	2	0	1	15200
2	2	0	12	1422
3	2	0	12	1057
4	4+	0	8	7969
...	...	...	...	...
550063	1	1	20	368
550064	3	0	20	371
550065	4+	1	20	137
550066	2	0	20	365
550067	4+	1	20	490

[550068 rows x 10 columns]

```
[20]: # shape of data
walmart.shape
```

[20]: (550068, 10)

No.of.rows = 550068 | No.of.columns = 10

Datatype of features

```
[21]: walmart.dtypes
```

```
[21]: User_ID          int64
Product_ID          object
Gender              object
Age                 object
Occupation           int64
City_Category        object
Stay_In_Current_City_Years  object
Marital_Status       int64
Product_Category     int64
Purchase             int64
dtype: object
```

- No.of categorical feature = 5 | No.of continuous feature = 5

Unique Value (counts) for each features

```
[22]: walmart.nunique()
```

```
[22]: User_ID          5891
      Product_ID     3631
      Gender          2
      Age             7
      Occupation     21
      City_Category   3
      Stay_In_Current_City_Years  5
      Marital_Status   2
      Product_Category 20
      Purchase       18105
      dtype: int64
```

Insights: - There are total 550068 datapoints out of which only 5891 User_ID found means same users bought multiple product or do multiple transactions.

Missing Values Detection

```
[23]: walmart.isnull().sum()
```

```
[23]: User_ID          0
      Product_ID      0
      Gender          0
      Age             0
      Occupation      0
      City_Category    0
      Stay_In_Current_City_Years  0
      Marital_Status   0
      Product_Category 0
      Purchase        0
      dtype: int64
```

- There is no missing values found in the dataset

Validating duplicate values

```
[24]: walmart.duplicated().sum()
```

```
[24]: np.int64(0)
```

There is no duplicated values found in the dataset

```
[25]: cat_col = ["User_ID", "Product_ID", "Gender", "Age", "City_Category",  
               ↪ "Marital_Status"]

      for i in cat_col:
          walmart[i] = walmart[i].astype("category")

      walmart.dtypes
```

```
[25]: User_ID          category
      Product_ID      category
      Gender          category
      Age            category
      Occupation      int64
      City_Category   category
      Stay_In_Current_City_Years  object
      Marital_Status  category
      Product_Category    int64
      Purchase          int64
      dtype: object
```

Statistical Summary

```
[26]: walmart.describe().T
```

```
[26]:
```

	count	mean	std	min	25%	50%	\
Occupation	550068.0	8.076707	6.522660	0.0	2.0	7.0	
Product_Category	550068.0	5.404270	3.936211	1.0	1.0	5.0	
Purchase	550068.0	9263.968713	5023.065394	12.0	5823.0	8047.0	

	75%	max
Occupation	14.0	20.0
Product_Category	8.0	20.0
Purchase	12054.0	23961.0

- As it is quite obvious that there is significant difference in the mean and std for Purchase column which might be containing outliers.

```
[27]: walmart.describe(include = ["object", "category"]).T
```

```
[27]:
```

	count	unique	top	freq
User_ID	550068	5891	1001680	1026
Product_ID	550068	3631	P00265242	1880
Gender	550068	2	M	414259
Age	550068	7	26-35	219587
City_Category	550068	3	B	231173
Stay_In_Current_City_Years	550068	5	1	193821
Marital_Status	550068	2	0	324731

- Customer (1001680) has purchased more than others
- Product (P00265242) is most bought item
- Most of the customers are Male
- Most of customers lies in [26-35] Age bracket
- Majority of the customers are Unmarried

Non-Graphical Analysis

Value counts and Unique Attributes

```
[28]: walmart.groupby(["Gender"])["User_ID"].nunique() / walmart["User_ID"].nunique()  
      ↪* 100
```

```
[28]: Gender  
      F    28.280428  
      M    71.719572  
      Name: User_ID, dtype: float64
```

```
[29]: walmart.groupby(["Gender"])["Purchase"].describe()
```

```
[29]:
```

	count	mean	std	min	25%	50%	75%	\
Gender								
F	135809.0	8734.565765	4767.233289	12.0	5433.0	7914.0	11400.0	
M	414259.0	9437.526040	5092.186210	12.0	5863.0	8098.0	12454.0	
		max						
Gender								
F	23959.0							
M	23961.0							

There are only 28% female customers. While male customers constitute around 72% in our dataset

```
[30]: walmart.groupby(["Age"])["User_ID"].nunique() / walmart["User_ID"].nunique() *  
      ↪100
```

```
[30]: Age  
      0-17    3.700560  
      18-25   18.146325  
      26-35   34.849771  
      36-45   19.809879  
      46-50    9.013750  
      51-55    8.164997  
      55+     6.314717  
      Name: User_ID, dtype: float64
```

```
[31]: walmart.groupby(["Age"])["Purchase"].describe()
```

```
[31]:
```

	count	mean	std	min	25%	50%	75%	\
Age								
0-17	15102.0	8933.464640	5111.114046	12.0	5328.0	7986.0	11874.0	
18-25	99660.0	9169.663606	5034.321997	12.0	5415.0	8027.0	12028.0	
26-35	219587.0	9252.690633	5010.527303	12.0	5475.0	8030.0	12047.0	
36-45	110013.0	9331.350695	5022.923879	12.0	5876.0	8061.0	12107.0	
46-50	45701.0	9208.625697	4967.216367	12.0	5888.0	8036.0	11997.0	
51-55	38501.0	9534.808031	5087.368080	12.0	6017.0	8130.0	12462.0	
55+	21504.0	9336.280459	5011.493996	12.0	6018.0	8105.5	11932.0	

	max
Age	
0-17	23955.0
18-25	23958.0
26-35	23961.0
36-45	23960.0
46-50	23960.0
51-55	23960.0
55+	23960.0

- Around 35% customers are in [26-35] age bracket. On other hand customers in age bracket [0-17], [51-55] & [55+] showing less purchasing behaviour.

```
[32]: walmart.groupby(["Stay_In_Current_City_Years"])["User_ID"].nunique()
```

```
[32]: Stay_In_Current_City_Years
0      772
1     2086
2     1145
3      979
4+      909
Name: User_ID, dtype: int64
```

- Customers staying for 1-2 years in same city purchased more than others

```
[33]: walmart.groupby(["Marital_Status"])["User_ID"].nunique() / walmart["User_ID"].
      ↪nunique() * 100
```

```
[33]: Marital_Status
0     58.003735
1     41.996265
Name: User_ID, dtype: float64
```

- 58:42 is the ratio between married and unmarried customers

```
[34]: walmart.groupby(["City_Category"])["User_ID"].nunique()
```

```
[34]: City_Category
A     1045
B     1707
C     3139
Name: User_ID, dtype: int64
```

- Most of customers belongs to C city_category

```
[35]: walmart.groupby(['City_Category'])['Purchase'].describe()
```

```
[35]:
```

	count	mean	std	min	25%	50% \
City_Category						
A	147720.0	8911.939216	4892.115238	12.0	5403.0	7931.0
B	231173.0	9151.300563	4955.496566	12.0	5460.0	8005.0
C	171175.0	9719.920993	5189.465121	12.0	6031.5	8585.0

	75%	max
City_Category		
A	11786.0	23961.0
B	11986.0	23960.0
C	13197.0	23961.0

- Most of the customers belongs to City C But customers from city_category B tend to purchase more suggest the fact they visit store multiple times during that period

Graphical Analysis

Univariate Countinous Variable Analysis

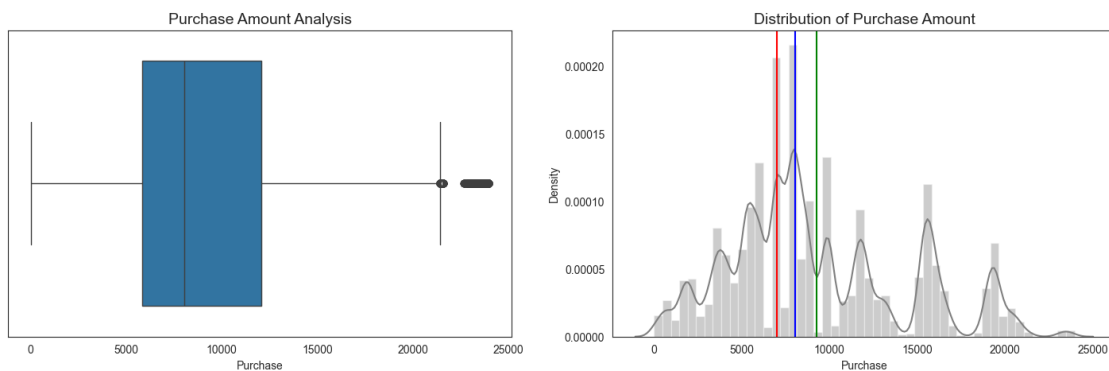
```
[38]: fig = plt.figure(figsize=(18,5))
sns.set_style("white")

plt.subplot(1, 2, 1)
sns.boxplot(data = walmart, x = "Purchase", orient = "h")

plt.title('Purchase Amount Analysis', fontsize = '14')

plt.subplot(1, 2, 2)
sns.distplot(a = walmart["Purchase"], color = 'gray')
plt.title("Distribution of Purchase Amount", fontsize = '14')
plt.axvline(walmart["Purchase"].mean(),color="g")
plt.axvline(walmart["Purchase"].median(),color="b")
plt.axvline(walmart["Purchase"].mode()[0],color="r")

plt.show()
```



Insights:

- There are outliers in purchase amount.
- While observing the distribution of purchase amount from density plot. It is quite obvious that the distribution is right skewed means majority of data concentrated on left side.
- Majority of customer purchase within 5,000 - 20,000 range.

Detecting Outliers

```
[39]: print('Mean of Purchase Amount  = ', walmart["Purchase"].mean())
      print('Median of Purchase Amount = ', walmart["Purchase"].median())
```

```
Mean of Purchase Amount  =  9263.968712959126
```

```
Median of Purchase Amount =  8047.0
```

- There is significant difference in Mean & Median of Purchase Amounts lead to outliers

Handling Outliers

```
[40]: q1 = walmart["Purchase"].quantile(0.25)
      q3 = walmart["Purchase"].quantile(0.75)

      IQR = q3 - q1
      print("1. First Quartile (q1) = ", q1)
      print("2. Third Quartile (q2) = ", q3)
      print("3. IQR                    = ", IQR)
```

```
1. First Quartile (q1) =  5823.0
```

```
2. Third Quartile (q2) =  12054.0
```

```
3. IQR                    =  6231.0
```

```
[41]: walmart_new = walmart.copy()

      # walmart_new = walmart_new.loc[(walmart_new["Purchase"] > q1 - 1.5*IQR) &
      ↪ (walmart_new["Purchase"] < q3 + 1.5*IQR)]
      walmart_new["Purchase"] = np.clip(walmart_new["Purchase"], q1-1.5*IQR, q3 + 1.
      ↪ 5*IQR)
      walmart_new.shape
```

```
[41]: (550068, 10)
```

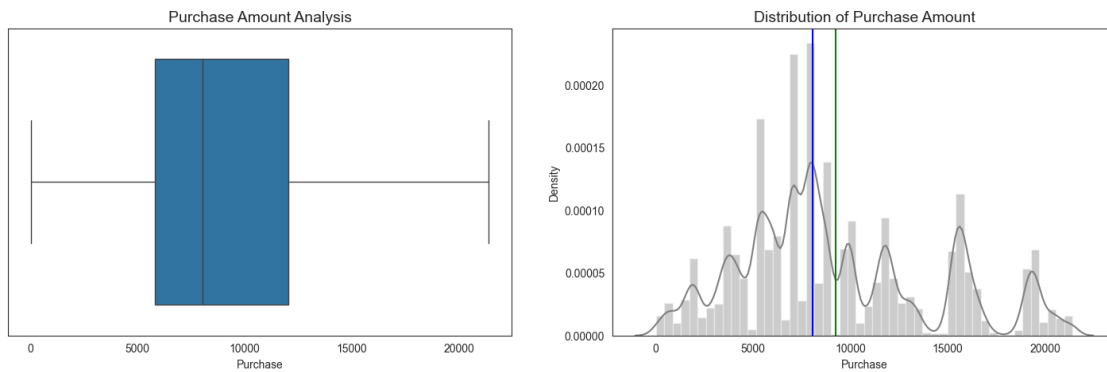
```
[42]: fig = plt.figure(figsize=(18,5))
      sns.set_style("white")

      plt.subplot(1, 2, 1)
      sns.boxplot(data = walmart_new, x = "Purchase", orient = "h")

      plt.title('Purchase Amount Analysis', fontsize = '14')
```

```
plt.subplot(1, 2, 2)
sns.distplot(a = walmart_new["Purchase"], color = 'gray')
plt.title("Distribution of Purchase Amount", fontsize = '14')
plt.axvline(walmart_new["Purchase"].mean(),color="g")
plt.axvline(walmart_new["Purchase"].median(),color="b")

plt.show()
```



- Outliers has been clipped now

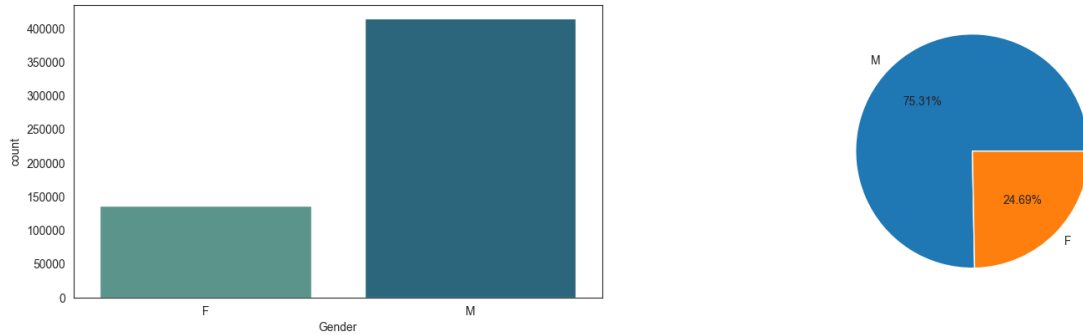
Univariate Categorical Variable Analysis

```
[43]: fig = plt.figure(figsize=(18,10))
sns.set_style(style='white')
ax1 = plt.subplot2grid((2,2),(0,0))
sns.countplot(data=walmart_new, x="Gender", palette="crest")

#first row sec column
ax1 = plt.subplot2grid((2,2), (0, 1))
plt.pie(walmart_new["Gender"].value_counts(),
        labels = walmart_new["Gender"].value_counts().index,
        autopct = '%1.2f%%')
plt.suptitle('Distribution of Gender', fontsize = 20)

plt.show()
```

Distribution of Gender



Insights:

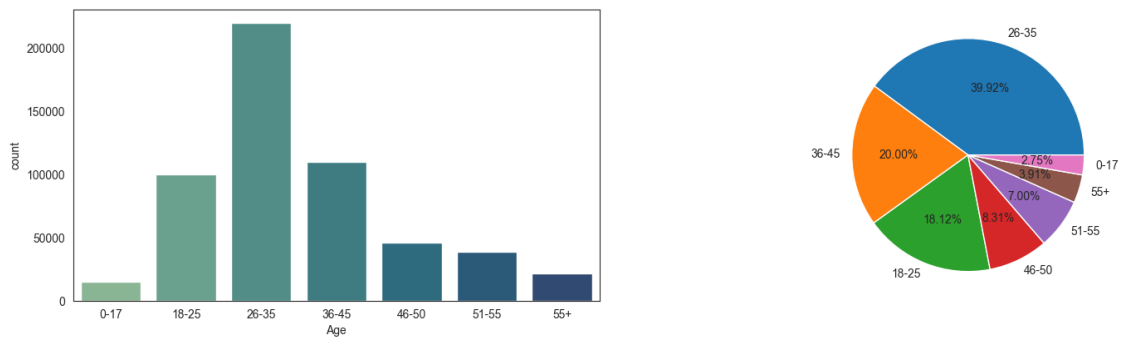
- Males clearly purchase more than females. 75% of men and only 25% of women purchase products.

```
[44]: fig = plt.figure(figsize=(18,10))
sns.set_style(style='white')
ax1 = plt.subplot2grid((2,2),(0,0))
sns.countplot(data=walmart_new, x="Age", palette="crest")

#first row sec column
ax1 = plt.subplot2grid((2,2), (0, 1))
plt.pie(walmart_new["Age"].value_counts(),
        labels = walmart_new["Age"].value_counts().index,
        autopct = '%1.2f%%')
plt.suptitle('Distribution of Age', fontsize = 20)

plt.show()
```

Distribution of Age



Insights:

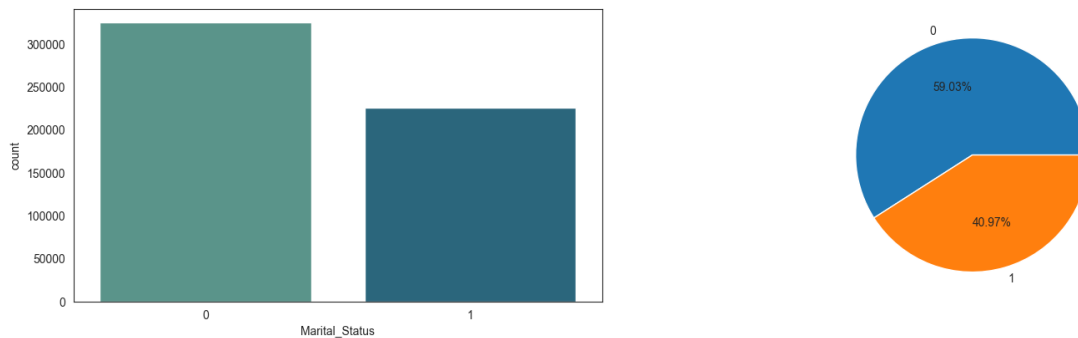
- 60% of purchases are made by people between the ages of 26 and 45

```
[45]: fig = plt.figure(figsize=(18,10))
sns.set_style(style='white')
ax1 = plt.subplot2grid((2,2),(0,0))
sns.countplot(data=walmart, x="Marital_Status", palette="crest")

#first row sec column
ax1 = plt.subplot2grid((2,2), (0, 1))
plt.pie(walmart_new["Marital_Status"].value_counts(),
        labels = walmart_new["Marital_Status"].value_counts().index,
        autopct = '%1.2f%%')
plt.suptitle('Distribution of Marital_Status', fontsize = 20)

plt.show()
```

Distribution of Marital_Status



Insights:

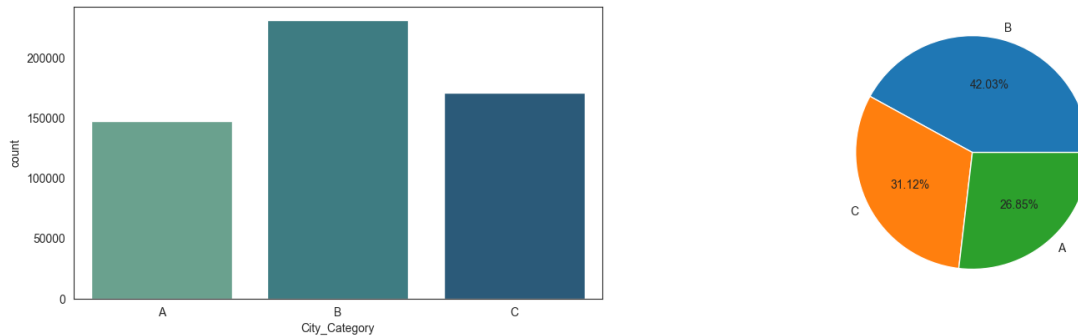
- Most of customers are unmarried.

```
[46]: fig = plt.figure(figsize=(18,10))
sns.set_style(style='white')
ax1 = plt.subplot2grid((2,2),(0,0))
sns.countplot(data=walmart_new, x="City_Category", palette="crest")

#first row sec column
ax1 = plt.subplot2grid((2,2), (0, 1))
plt.pie(walmart_new["City_Category"].value_counts(),
        labels = walmart_new["City_Category"].value_counts().index,
        autopct = '%1.2f%%')
plt.suptitle('Distribution of City_Category', fontsize = 20)
```

```
plt.show()
```

Distribution of City_Category



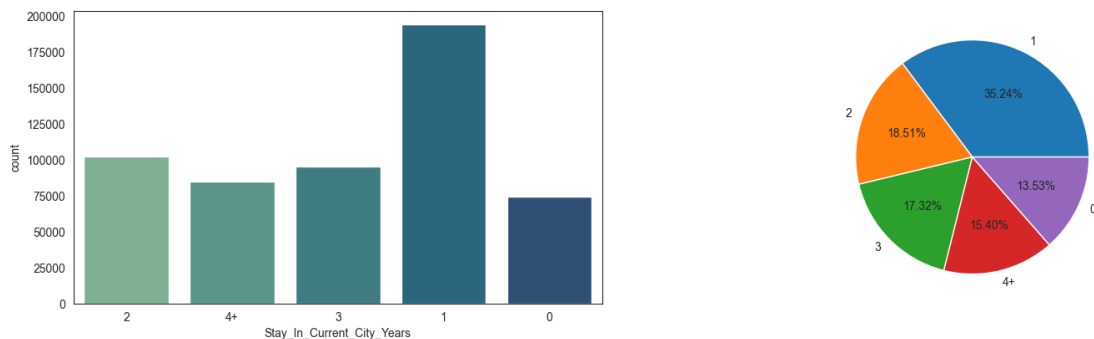
Insights:

- City Category B accounts for 42%, City Category C 31%, and City Category A represents 27% of all customer purchases.

```
[47]: fig = plt.figure(figsize=(18,10))
sns.set_style(style='white')
ax1 = plt.subplot2grid((2,2),(0,0))
sns.countplot(data=walmart_new, x="Stay_In_Current_City_Years", palette="crest")

#first row sec column
ax1 = plt.subplot2grid((2,2), (0, 1))
plt.pie(walmart_new["Stay_In_Current_City_Years"].value_counts(),
        labels = walmart_new["Stay_In_Current_City_Years"].value_counts().index,
        autopct = '%1.2f%%')
plt.suptitle('Distribution of Stay_In_Current_City_Years', fontsize = 20)
plt.show()
```

Distribution of Stay_In_Current_City_Years

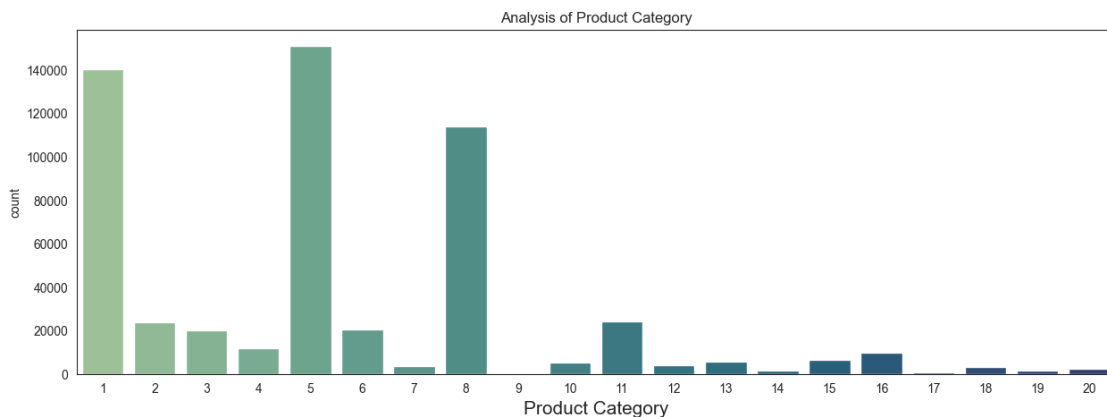


Insights:

- Most of customer stayed around 1 years in the current city

```
[48]: fig = plt.figure(figsize = (15,5))

sns.countplot(data = walmart_new, x = "Product_Category", palette="crest")
plt.xlabel("Product Category", fontsize = 15)
plt.title("Analysis of Product Category")
plt.show()
```



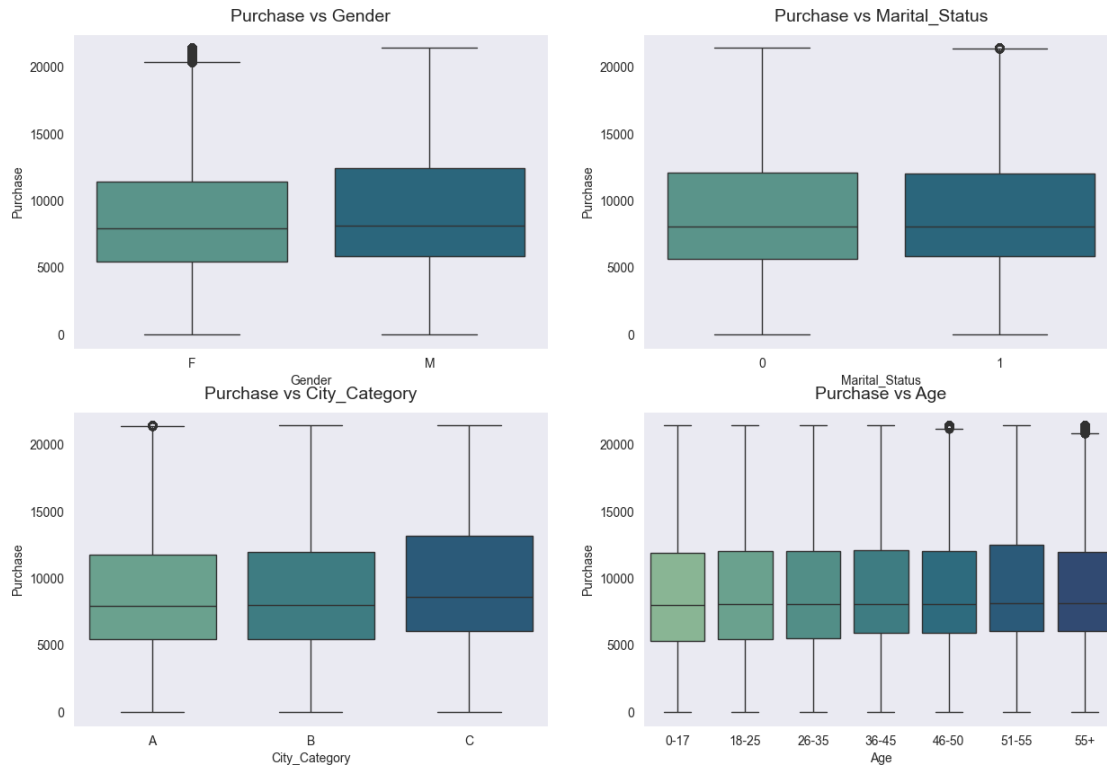
Insights:

- Product categories 1, 5 and 8 most bought products.

Bivariate Analysis

```
[50]: cat_col = ["Gender", "Marital_Status", "City_Category", "Age"]

fig, axs = plt.subplots(nrows=2, ncols = 2, figsize=(15,10))
k = 0
sns.set_style("dark")
for i in range(2):
    for j in range(2):
        sns.boxplot(data=walmart_new, x=cat_col[k], y="Purchase",
                    palette='crest', ax=axs[i, j])
        axs[i, j].set_title("Purchase vs " + cat_col[k], pad = 10, fontsize =
                    14)
        k += 1
plt.show()
```



Insights:

- Customers purchasing behaviour are almost stable at equilibrium irrespective on Gender, Marital_status, City_Category & Age
- Median purchase from each features are almost same

Lets draw analysis for Gender

```
[52]: fig = plt.figure(figsize=(18, 15))

sns.set_style("white")

ax1 = plt.subplot2grid((2, 2), (0, 0))
sns.barplot(data=walmart_new, x = "Age", y = "Purchase", ax=ax1, hue = "Gender", palette="crest")
plt.title("Gender Analysis Purchase vs Age", pad = 10, fontsize = 14)

ax1 = plt.subplot2grid((2, 2), (0, 1))
sns.barplot(data=walmart_new, x = "Marital_Status", y = "Purchase", ax=ax1, hue = "Gender", palette="crest")
plt.title("Gender Analysis Purchase vs Marital Status", pad = 10, fontsize = 14)

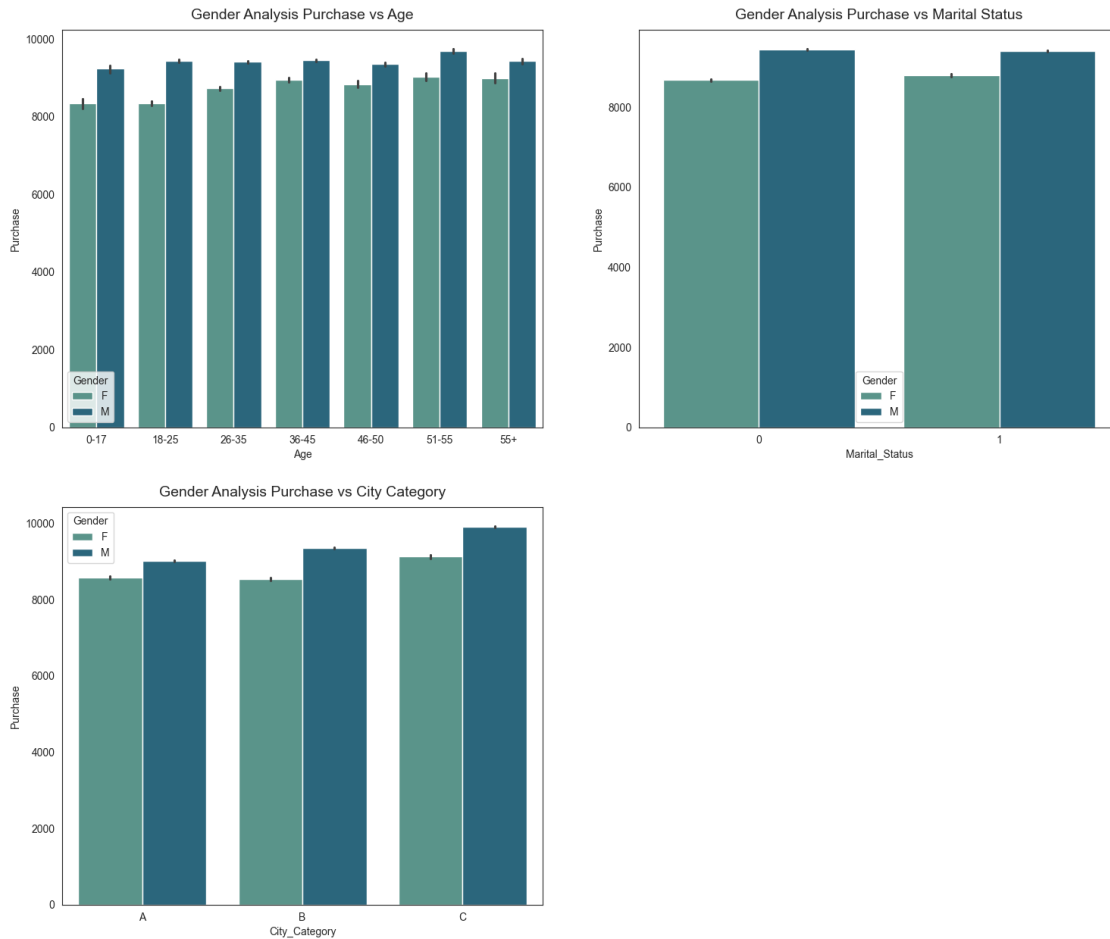
ax1 = plt.subplot2grid((2, 2), (1, 0))
```

```

sns.barplot(data=walmart_new, x = "City_Category", y = "Purchase", ax=ax1, hue_
↪   "Gender", palette="crest")
plt.title("Gender Analysis Purchase vs City Category", pad = 10, fontsize = 14)

plt.show()

```



Insights:

- Purchases are high in city category C
- Purchase is the somehow same for all age groups
- Female Customers form City category C is more than other city category
- Purchases are same for all Marital Status

```

[53]: fig = plt.figure(figsize=(18, 15))

sns.set_style("white")

```

```

ax1 = plt.subplot2grid((2, 2), (0, 0))
sns.countplot(data=walmart_new, x = "Age", ax=ax1, hue = "Gender",
               palette="crest")
plt.title("Age Churn Status", pad = 10, fontsize = 14)

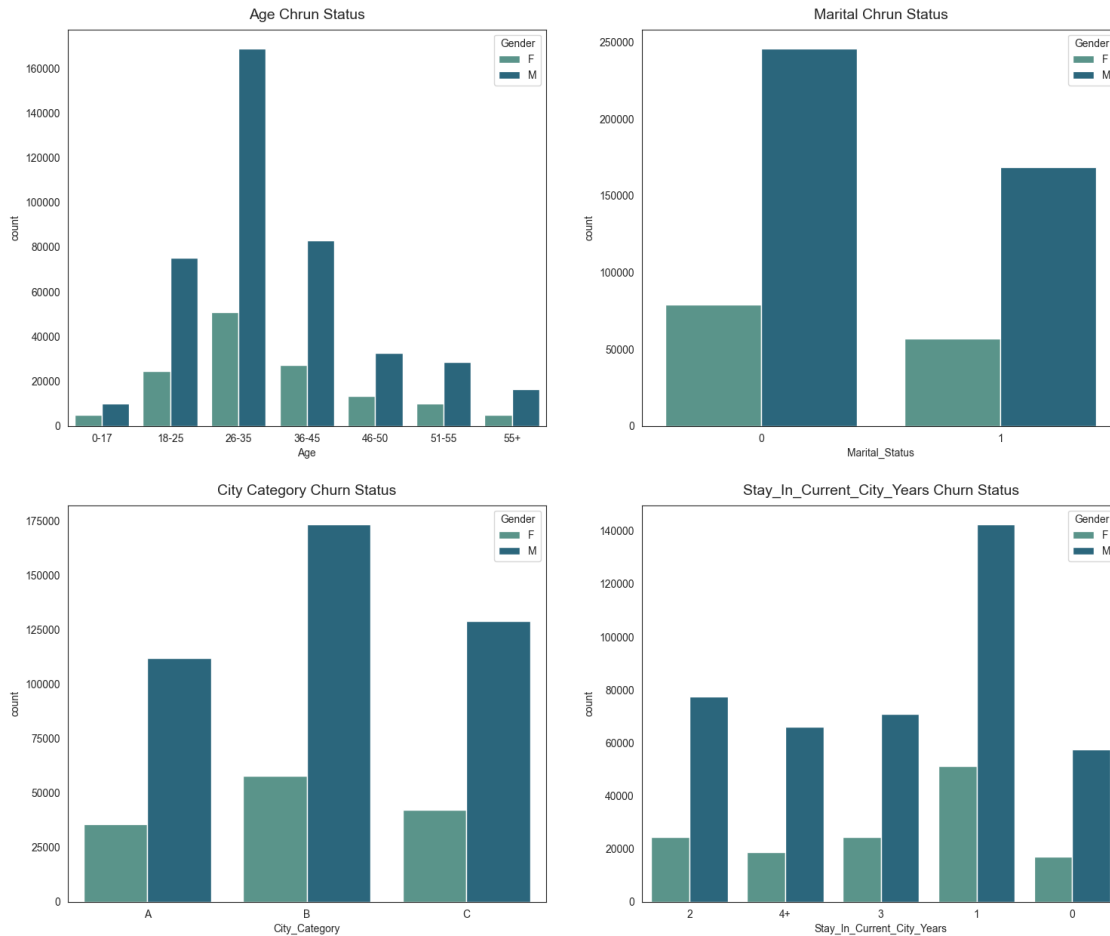
ax1 = plt.subplot2grid((2, 2), (0, 1))
sns.countplot(data=walmart_new, x = "Marital_Status", ax=ax1, hue = "Gender",
               palette="crest")
plt.title("Marital Churn Status", pad = 10, fontsize = 14)

ax1 = plt.subplot2grid((2, 2), (1, 0))
sns.countplot(data=walmart_new, x = "City_Category", ax=ax1, hue = "Gender",
               palette="crest")
plt.title("City Category Churn Status", pad = 10, fontsize = 14)

ax1 = plt.subplot2grid((2, 2), (1, 1))
sns.countplot(data=walmart_new, x = "Stay_In_Current_City_Years", ax=ax1, hue =
               "Gender", palette="crest")
plt.title("Stay_In_Current_City_Years Churn Status", pad = 10, fontsize = 14)

plt.show()

```



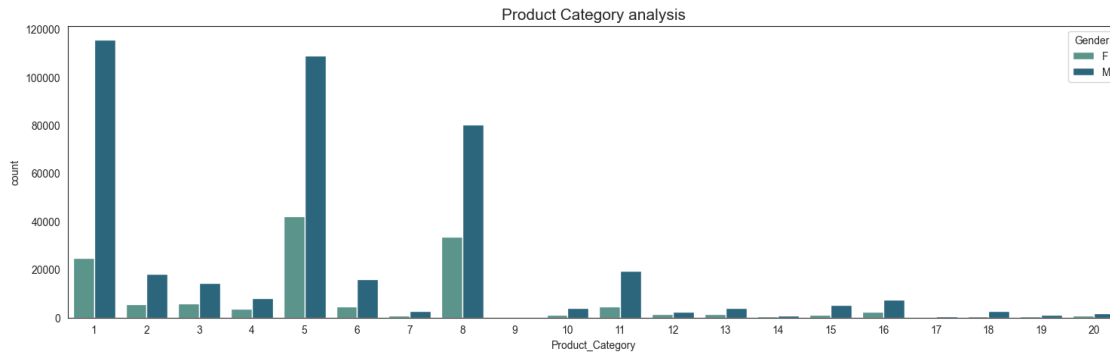
Insights: - Most of female customers are from city category C

- Most of female customer stay 1 year in the current city than others
- Most of the customers are in range 18-45 age bracket
- There is huge difference in the male and female customer for age bracket [26-35]
- There is significant difference of male customers. Most of the male customers are single. While female customers marital status not more significant as there is not so much difference.

```
[54]: fig = plt.figure(figsize=(18, 5))

plt.title("Product Category analysis", fontsize = 15)
sns.countplot(data=walmart_new, x = "Product_Category", hue = "Gender",
              palette="crest")

plt.show()
```



Insights: - Product Category 5 and 8 are demanding among female customers

- Product Category 1 and 5 are most demanding among male customers

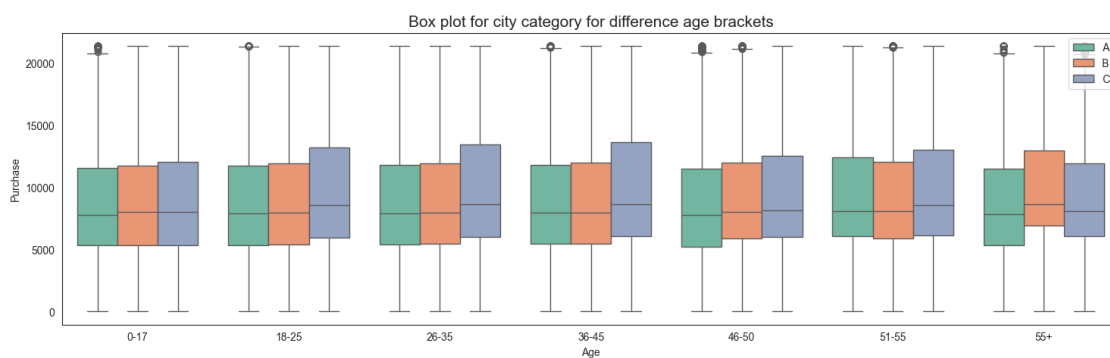
Lets draw analysis for City Category Among Different Age Brackets

```
[55]: fig = plt.figure(figsize=(18, 5))

plt.title("Box plot for city category for difference age brackets", fontsize = 15)

sns.boxplot(data=walmart_new, x = "Age", y = "Purchase", hue = "City_Category", palette="Set2")

plt.legend(loc = "upper right")
plt.show()
```



Insights: - Product Category B is popular among age group 55+

- Product Category A & B are quite same in demand among other age groups except 55+
- While Product Category C is in demand for age group 18-45 and slightly in 51-55 age group

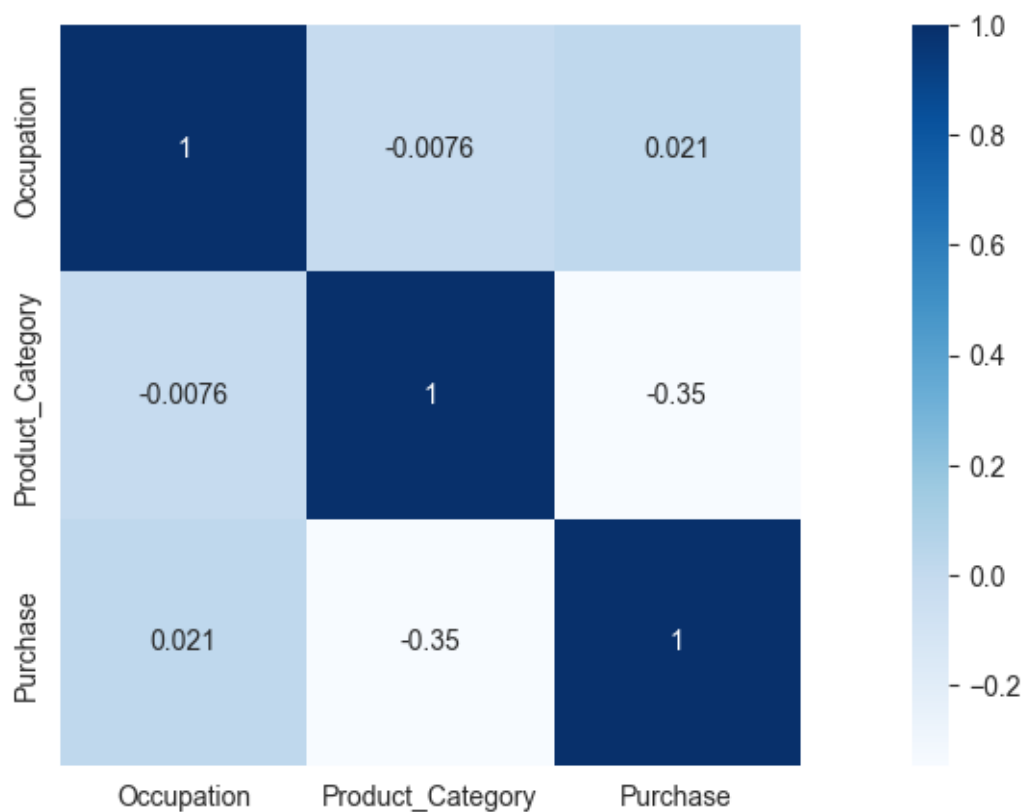
Correlation Analysis

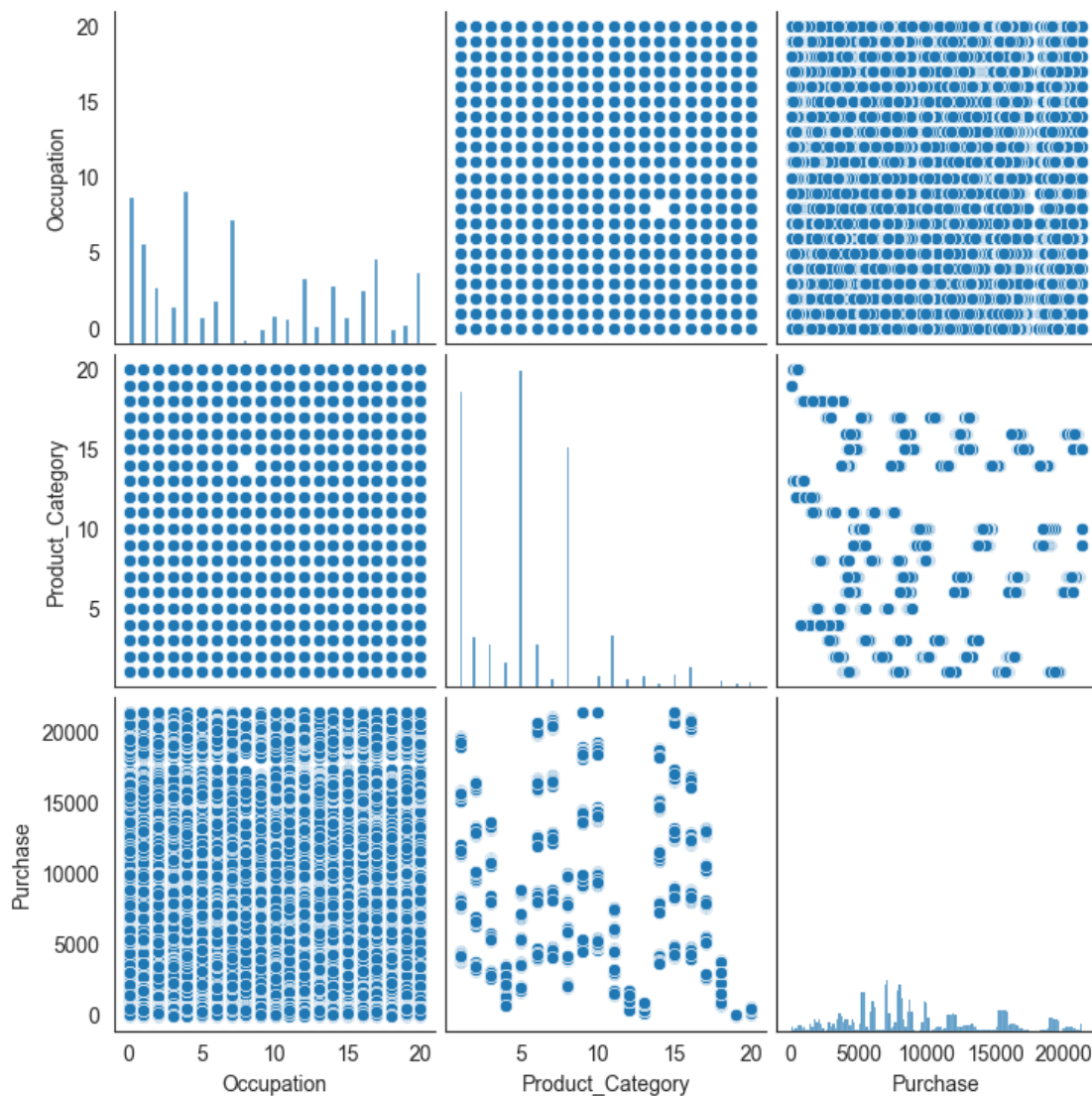
```
[58]: walmart_new.corr(numeric_only=True)
```

```
[58]:
```

	Occupation	Product_Category	Purchase
Occupation	1.000000	-0.007618	0.020853
Product_Category	-0.007618	1.000000	-0.347413
Purchase	0.020853	-0.347413	1.000000

```
[60]: plt.figure(figsize = (15, 5))  
sns.heatmap(data=walmart_new.corr(numeric_only=True), annot=True, cmap="Blues",  
            square=True)  
  
sns.pairplot(walmart_new)  
plt.show()
```





Insights: - Our dataset is highly categorical variable centric. So we can hardly find correlation in our dataset.

4. Answering questions

4.1 Are women spending more money per transaction than men? Why or Why not?

```
[61]: walmart_new.groupby(["Gender", "City_Category"])["User_ID"].count()
```

```
[61]: Gender  City_Category
      F         A          35704
      F         B          57796
      F         C          42309
      M         A          112016
```

```

B          173377
C          128866
Name: User_ID, dtype: int64

```

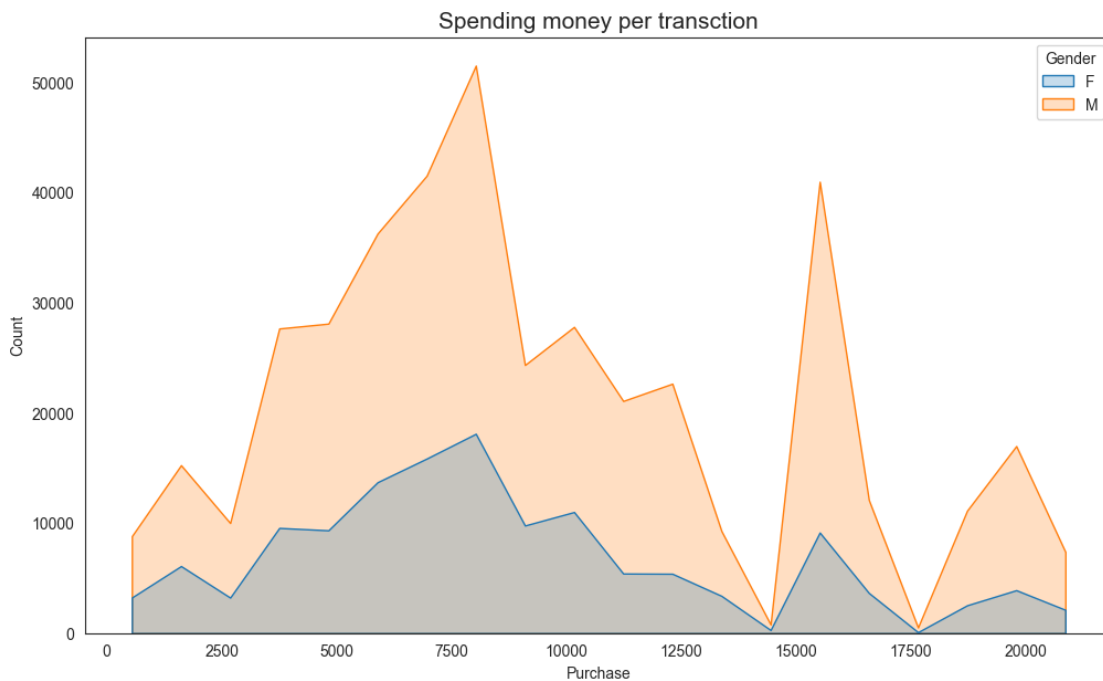
```

[62]: plt.figure(figsize=(12, 7))

plt.title("Spending money per transection", fontsize = 15)
sns.histplot(data=walmart_new, x = "Purchase", bins=20, hue = "Gender",
             element="poly")

plt.show()

```



Insights: - The amount of money spent by women customers per transaction is quite less than that of men. There would be multiple reasons for that : - Socio-economic status

- Generally male starts earning way before females
- Generally male customers earns more than females

Lets take sample of data to verify this fact

```

[66]: walmart_new.sample(5000, replace=True).groupby(["Gender"])["Purchase"].
      describe()

```

```

[66]:      count      mean      std      min      25%      50%      75%  \
Gender
F      1252.0  8991.064696  4809.536667  118.0  5844.5  7952.0  11732.75

```

```

M          3748.0  9278.515608  5027.291839   12.0  5716.5  8086.5  12057.00

          max
Gender
F          21400.5
M          21400.5

```

Insights: - Even after taking sample of 5000 customer, male customers tends to spend more than that of females

4.2 Confidence intervals and distribution of the mean of the expenses by female and male customers

```
[67]: walmart_new.groupby(["Gender"])["Purchase"].describe()
```

```

[67]:      count      mean      std   min   25%   50%   75%  \
Gender
F    135809.0  8726.256327  4743.242716  12.0  5433.0  7914.0  11400.0
M    414259.0  9428.373455  5068.732811  12.0  5863.0  8098.0  12454.0

          max
Gender
F          21400.5
M          21400.5

```

```
[68]: walmart_new.shape
```

```
[68]: (550068, 10)
```

Insights: - After clipping outlier in our dataset. We have found that out of 550k datapoint -

- Males are 414K
- Females are 135K

```

[71]: walmart_new_smp_male = walmart_new.loc[walmart["Gender"] == "M"]["Purchase"]
      walmart_new_smp_female = walmart_new.loc[walmart["Gender"] == "F"]["Purchase"]

```

```

[72]: print("Male customers are: ", walmart_new_smp_male.shape[0])
      print("Female customers are: ", walmart_new_smp_female.shape[0])

```

```

Male customers are:  414259
Female customers are:  135809

```

Calculating CI (90%, 95%, 99%) using Bootstrapping for Purchases based on Gender using CLT

```

[90]: #creating a function to calculate confidence interval

def confidence_interval(data,ci):
    #converting the list to series
    l_ci = (100-ci)/2
    u_ci = (100+ci)/2

```

```

#calculating lower limit and upper limit of confidence interval
interval = np.percentile(data,[l_ci,u_ci]).round(0)

return interval

```

```

[91]: #defining a function for plotting the visual for given confidence interval

def plot(ci):

    #setting the plot style
    fig = plt.figure(figsize = (15,8))
    gs = fig.add_gridspec(2,2)

    #creating separate data frames for each gender
    df_male = df.loc[df['Gender'] == 'M', 'Purchase']
    df_female = df.loc[df['Gender'] == 'F', 'Purchase']

    #sample sizes and corresponding plot positions
    sample_sizes = [(100,0,0),(1000,0,1),(5000,1,0),(50000,1,1)]

    #number of samples to be taken from purchase amount
    bootstrap_samples = 20000

    male_samples = {}
    female_samples = {}

    for i,x,y in sample_sizes:
        male_means = [] #list for collecting the means of male sample
        female_means = [] #list for collecting the means of female sample

        for j in range(bootstrap_samples):

            #creating random 5000 samples of i sample size
            male_bootstrapped_samples = np.random.choice(df_male,size = i)
            female_bootstrapped_samples = np.random.choice(df_female,size = i)

            #calculating mean of those samples
            male_sample_mean = np.mean(male_bootstrapped_samples)
            female_sample_mean = np.mean(female_bootstrapped_samples)

            #appending the mean to the list
            male_means.append(male_sample_mean)
            female_means.append(female_sample_mean)

        #storing the above sample generated
        male_samples[f'{ci}%_{i}'] = male_means

```

```

female_samples[f'{ci}%_{i}'] = female_means

#creating a temporary dataframe for creating kdeplot
temp_df = pd.DataFrame(data = {'male_means':male_means, 'female_means':
↪female_means})

                                                                    #plotting kdeplots

#plot position
ax = fig.add_subplot(gs[x,y])

#plots for male and female
sns.kdeplot(data = temp_df,x = 'male_means',color = "#3A7089" ,fill =_
↪True, alpha = 0.5,ax = ax,label = 'Male')
sns.kdeplot(data = temp_df,x = 'female_means',color = "#4b4b4c" ,fill =_
↪True, alpha = 0.5,ax = ax,label = 'Female')

#calculating confidence intervals for given confidence level(ci)
m_range = confidence_interval(male_means,ci)
f_range = confidence_interval(female_means,ci)

#plotting confidence interval on the distribution
for k in m_range:
    ax.axvline(x = k,ymax = 0.9, color = "#3A7089",linestyle = '--')

for k in f_range:
    ax.axvline(x = k,ymax = 0.9, color = "#4b4b4c",linestyle = '--')

#removing the axis lines
for s in ['top','left','right']:
    ax.spines[s].set_visible(False)

# adjusting axis labels
ax.set_yticks([])
ax.set_ylabel('')
ax.set_xlabel('')

#setting title for visual
ax.set_title(f'CLT Curve for Sample Size = {i}',{'font':'serif', 'size':
↪11,'weight':'bold'})

plt.legend()

#setting title for visual
fig.suptitle(f'{ci}% Confidence Interval',font = 'serif', size = 18, weight_
↪= 'bold')

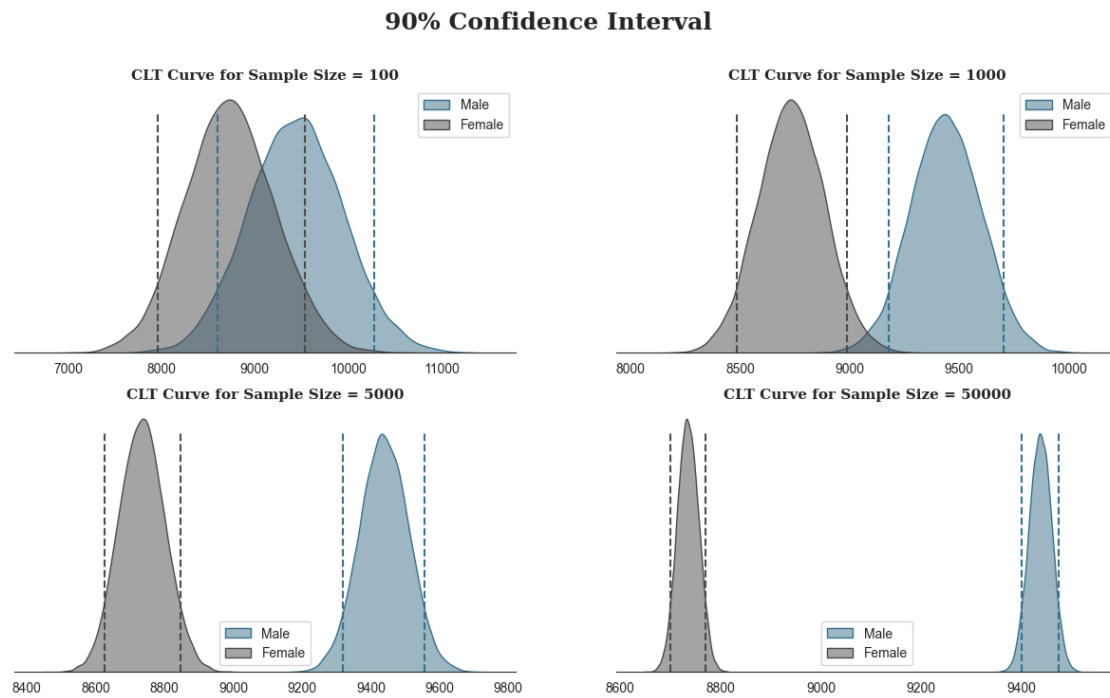
```

```
plt.show()

return male_samples,female_samples
```

Calculation 90% CI

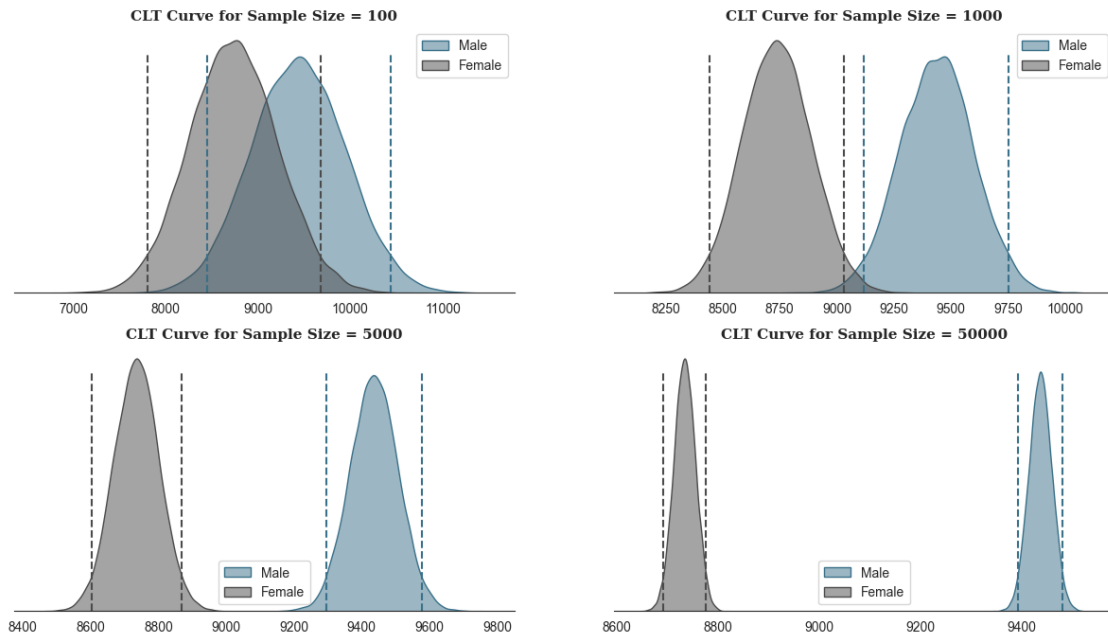
```
[96]: m_samp_90,f_samp_90 = plot(90)
```



Calculation 95% CI

```
[97]: m_samp_95,f_samp_95 = plot(95)
```

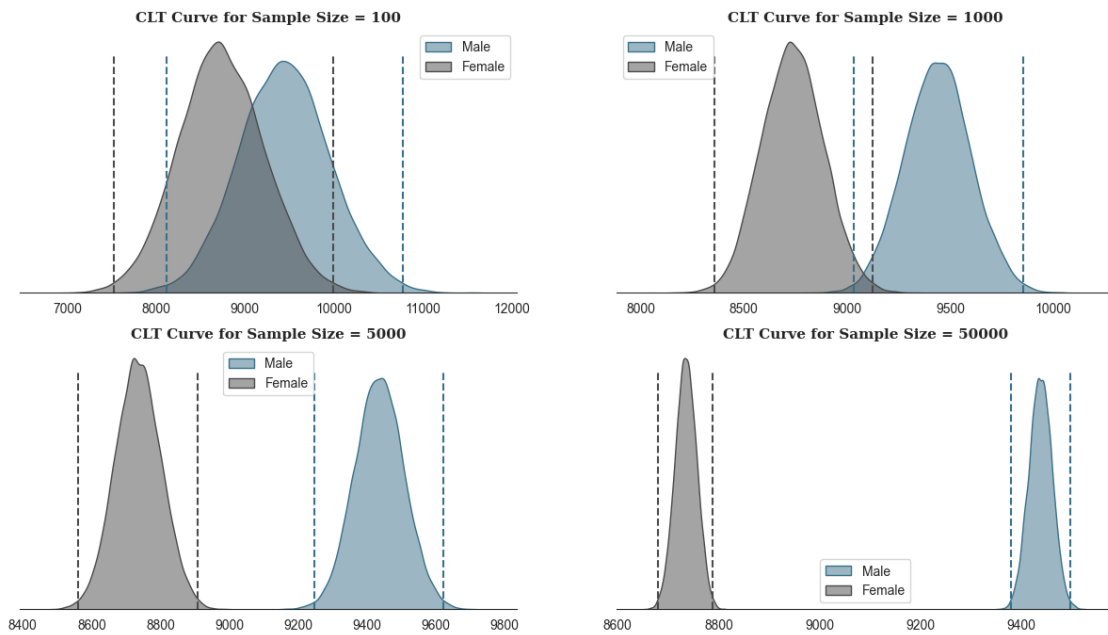
95% Confidence Interval



Calculation 99% CI

```
[100]: m_samp_99,f_samp_99 = plot(99)
```

99% Confidence Interval



4.3 Are confidence intervals of average male and female spending overlapping?

```
[101]: fig = plt.figure(figsize = (20,10))
gs = fig.add_gridspec(3,1)

for i,j,k,l in_
    [(m_samp_90,f_samp_90,90,0),(m_samp_95,f_samp_95,95,1),(m_samp_99,f_samp_99,99,2)]:
    #list for collecting ci for given cl
    m_ci = ['Male']
    f_ci = ['Female']

    #finding ci for each sample size (males)
    for m in i:
        m_range = confidence_interval(i[m],k)
        m_ci.append(f"CI = ${m_range[0]:.0f} - ${m_range[1]:.0f}, Range =_
    [(m_range[1] - m_range[0]):.0f}")

    #finding ci for each sample size (females)
    for f in j:
        f_range = confidence_interval(j[f],k)
        f_ci.append(f"CI = ${f_range[0]:.0f} - ${f_range[1]:.0f}, Range =_
    [(f_range[1] - f_range[0]):.0f]")

    #plotting the summary

    ax = fig.add_subplot(gs[l])

    #contents of the table
    ci_info = [m_ci,f_ci]

    #plotting the table
    table = ax.table(cellText = ci_info, cellLoc='center',
                    colLabels = ['Gender','Sample Size = 100','Sample Size =_
    1000','Sample Size = 5000','Sample Size = 50000'],
                    colLoc = 'center',colWidths = [0.05,0.2375,0.2375,0.2375,0.
    2375],bbox = [0, 0, 1, 1])

    table.set_fontsize(13)

    #removing axis
    ax.axis('off')

    #setting title
    ax.set_title(f"{k}% Confidence Interval Summary",{ 'font':'serif', 'size':
    14, 'weight':'bold'})
```

90% Confidence Interval Summary				
Gender	Sample Size = 100	Sample Size = 1000	Sample Size = 5000	Sample Size = 50000
Male	CI = 8605 – 10272, Range = 1667	CI = 9180 – 9704, Range = 524	CI = 9320 – 9556, Range = 236	CI = 9400 – 9475, Range = 75
Female	CI = 7960 – 9537, Range = 1577	CI = 8485 – 8986, Range = 501	CI = 8625 – 8847, Range = 222	CI = 8700 – 8770, Range = 70

95% Confidence Interval Summary				
Gender	Sample Size = 100	Sample Size = 1000	Sample Size = 5000	Sample Size = 50000
Male	CI = 8451 – 10438, Range = 1987	CI = 9119 – 9754, Range = 635	CI = 9297 – 9578, Range = 281	CI = 9393 – 9482, Range = 89
Female	CI = 7809 – 9679, Range = 1870	CI = 8440 – 9033, Range = 593	CI = 8602 – 8867, Range = 265	CI = 8692 – 8776, Range = 84

99% Confidence Interval Summary				
Gender	Sample Size = 100	Sample Size = 1000	Sample Size = 5000	Sample Size = 50000
Male	CI = 8119 – 10781, Range = 2662	CI = 9032 – 9855, Range = 823	CI = 9248 – 9624, Range = 376	CI = 9380 – 9497, Range = 117
Female	CI = 7527 – 9998, Range = 2471	CI = 8357 – 9125, Range = 768	CI = 8561 – 8908, Range = 347	CI = 8680 – 8788, Range = 108

Insights:

1. Confidence Intervals

- From the above analysis, we can see that except for the Sample Size of 100, the confidence interval do not overlap as the sample size increases. This means that there is a statistically significant difference between the average spending per transaction for men and women within the given samples

2. Women spend less

- Men tend to spend more money per transaction on average than women, as the upper bounds of the confidence intervals for men are consistently higher than those for women across different sample sizes.

5. How can Walmart leverage this conclusion to make changes or improvements?

5.1. Segmentation Opportunities

- Walmart can create targeted marketing campaigns, loyalty programs, or product bundles to cater to the distinct spending behaviors of male and female customers. This approach may help maximize revenue from each customer segment.

5.2. Pricing Strategies

- Based on the above data of average spending per transaction by gender, they might adjust pricing or discount strategies to incentivize higher spending among male customers while ensuring competitive pricing for female-oriented products.

Data Visualization

```
[119]: #creating a df for purchase amount vs marital status
```

```
temp = df.groupby('Marital_Status')['Purchase'].agg(['sum','count']).
    ↪reset_index()
```

```
#calculating the amount in billions
```

```
temp['sum_in_billions'] = round(temp['sum'] / 10**9,2)
```

```
#calculating percentage distribution of purchase amount
```

```
temp['%sum'] = round(temp['sum']/temp['sum'].sum(),3)
```

```
#calculating per purchase amount
```

```
temp['per_purchase'] = round(temp['sum']/temp['count'])
```

```
temp
```

```
[119]:
```

	Marital_Status	sum	count	sum_in_billions	%sum	per_purchase
0	0	3008927447	324731	3.01	0.59	9266.0
1	1	2086885295	225337	2.09	0.41	9261.0

```
[125]: #setting the plot style
```

```
fig = plt.figure(figsize = (15,14))
```

```
gs = fig.add_gridspec(3,2,height_ratios =[0.10,0.4,0.5])
```

```
#Distribution of Purchase Amount
```

```
ax = fig.add_subplot(gs[0,:])
```

```
#plotting the visual
```

```
ax.barh(temp.loc[0,'Marital_Status'],width = temp.loc[0,'%sum'],color = '
    ↪"#3A7089",label = 'Unmarried')
```

```
ax.barh(temp.loc[1,'Marital_Status'],width = temp.loc[1,'%sum'],left =temp.
    ↪loc[0,'%sum'], color = "#4b4b4c",label = 'Married')
```

```
#inserting the text
```

```
txt = [0.0] #for left parameter in ax.text()
```

```
for i in temp.index:
```

```
    #for amount
```

```
    ax.text(temp.loc[i,'%sum']/2 + txt[0],0.15,f"${temp.
    ↪loc[i,'sum_in_billions']} Billion",
            va = 'center', ha='center',fontsize=18, color='white')
```

```
    #for marital status
```

```
    ax.text(temp.loc[i,'%sum']/2 + txt[0],- 0.20 ,f"{temp.
    ↪loc[i,'Marital_Status']}",
            va = 'center', ha='center',fontsize=14, color='white')
```

```
    txt += temp.loc[i,'%sum']
```

```

#removing the axis lines
for s in ['top','left','right','bottom']:
    ax.spines[s].set_visible(False)

#customizing ticks
ax.set_xticks([])
ax.set_yticks([])
ax.set_xlim(0,1)

#plot title
ax.set_title('Marital_Status-Based Purchase Amount Distribution',{'font':
    ↳'serif', 'size':15,'weight':'bold'})

#Distribution of Purchase Amount
↳per Transaction

ax1 = fig.add_subplot(gs[1,0])

color_map = ["#3A7089", "#4b4b4c"]

#plotting the visual
ax1.bar(temp['Marital_Status'],temp['per_purchase'],color = color_map,zorder =
↳2,width = 0.3)

#adding average transaction line
avg = round(df['Purchase'].mean())

ax1.axhline(y = avg, color = 'red', zorder = 0,linestyle = '--')

#adding text for the line
ax1.text(0.4,avg + 300, f"Avg. Transaction Amount ${avg:.0f}",
    {'font':'serif','size' : 12},ha = 'center',va = 'center')

#adjusting the ylimits
ax1.set_ylim(0,11000)

#adding the value_counts
for i in temp.index:
    ax1.text(temp.loc[i,'Marital_Status'],temp.loc[i,'per_purchase']/2,f"${temp.
↳loc[i,'per_purchase']:.0f}",
        {'font':'serif','size' : 12,'color':'white','weight':'bold' },ha =
↳'center',va = 'center')

#adding grid lines

```

```

ax1.grid(color = 'black',linestyle = '--', axis = 'y', zorder = 0, dashes =
↳(5,10))

#removing the axis lines
for s in ['top','left','right']:
    ax1.spines[s].set_visible(False)

#adding axis label
ax1.set_ylabel('Purchase Amount',fontweight = 'bold',fontsize = 12)
ax1.set_xticklabels(temp['Marital_Status'],fontweight = 'bold',fontsize = 12)

#setting title for visual
ax1.set_title('Average Purchase Amount per Transaction',{'font':'serif', 'size':
↳15,'weight':'bold'})

# creating pie chart for Marital_Status
↳distribution
ax2 = fig.add_subplot(gs[1,1])

color_map = ["#3A7089", "#4b4b4c"]
ax2.pie(temp['count'],labels = temp['Marital_Status'],autopct = '%.1f%%',
        shadow = True,colors = color_map,wedgeprops = {'linewidth':
↳5},textprops={'fontsize': 13, 'color': 'black'})

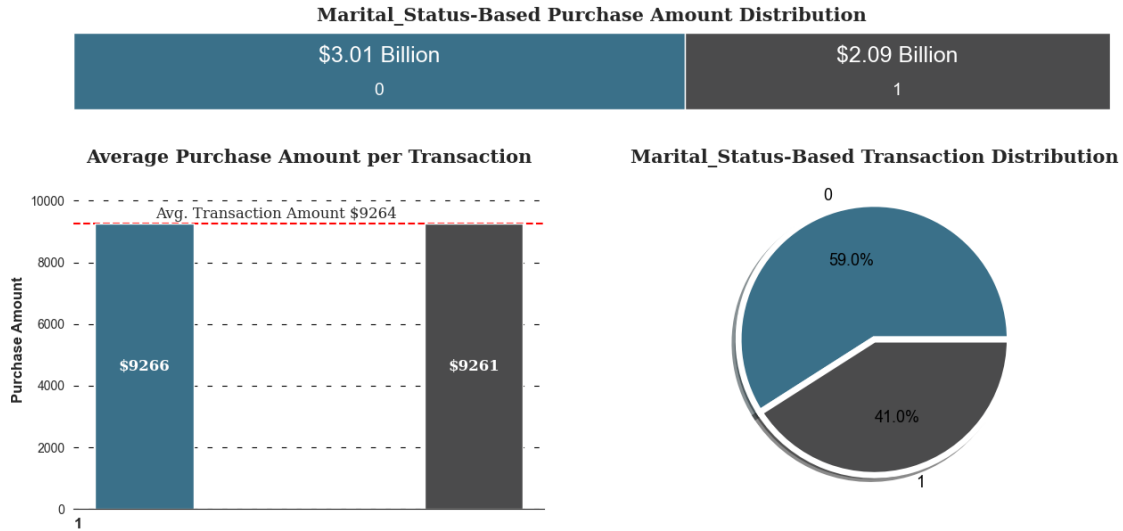
#setting title for visual
ax2.set_title('Marital_Status-Based Transaction Distribution',{'font':'serif',
↳'size':15,'weight':'bold'})

# adjusting axis labels
ax3.set_yticks([])
ax3.set_ylabel('')
ax3.set_xlabel('Purchase Amount',fontweight = 'bold',fontsize = 12)

#setting title for visual
ax3.set_title('Purchase Amount Distribution by Marital_Status',{'font':'serif',
↳'size':15,'weight':'bold'})

plt.show()

```



Insights:

1. Total Sales and Transactions Comparison - The total purchase amount and number of transactions by Unmarried customers was more than 20% the amount and transactions by married customers indicating that they had a more significant impact on the Black Friday sales.
2. Average Transaction Value - The average purchase amount per transaction was almost similar for married and unmarried customers (\$9261 vs \$9266)
3. Distribution of Purchase Amount - As seen above, the purchase amount for both married and unmarried customers is not normally distributed.

Confidence Interval Construction: Estimating Average Purchase Amount per Transaction

1. Step 1 - Building CLT Curve

- As seen above, the purchase amount distribution is not Normal. So we need to use Central Limit Theorem. It states the distribution of sample means will approximate a normal distribution, regardless of the underlying population distribution
- 2. Step 2 - Building Confidence Interval
- After building CLT curve, we will create a confidence interval predicting population mean at 95% Confidence level.

Note - We will use different sample sizes of [100,1000,5000,50000]

```
[130]: #creating a df for purchase amount vs age group
temp = df.groupby('Age')['Purchase'].agg(['sum', 'count']).reset_index()

#calculating the amount in billions
temp['sum_in_billions'] = round(temp['sum'] / 10**9, 2)

#calculating percentage distribution of purchase amount
```

```
temp['%sum'] = round(temp['sum']/temp['sum'].sum(),3)

#calculationg per purchase amount
temp['per_purchase'] = round(temp['sum']/temp['count'])

temp
```

```
[130]:
```

	Age	sum	count	sum_in_billions	%sum	per_purchase
0	0-17	134913183	15102	0.13	0.026	8933.0
1	18-25	913848675	99660	0.91	0.179	9170.0
2	26-35	2031770578	219587	2.03	0.399	9253.0
3	36-45	1026569884	110013	1.03	0.201	9331.0
4	46-50	420843403	45701	0.42	0.083	9209.0
5	51-55	367099644	38501	0.37	0.072	9535.0
6	55+	200767375	21504	0.20	0.039	9336.0

```
[131]: #setting the plot style
fig = plt.figure(figsize = (20,14))
gs = fig.add_gridspec(3,1,height_ratios =[0.10,0.4,0.5])

#Distribution of Purchase Amount

ax = fig.add_subplot(gs[0])
color_map = ["#3A7089",
↳ "#4b4b4c", '#99AEbB', '#5C8374', '#6F7597', '#7A9D54', '#9EB384']

#plotting the visual
left = 0

for i in temp.index:
    ax.barh(temp.loc[0,'Age'],width = temp.loc[i,'%sum'],left = left,color =
↳ color_map[i],label = temp.loc[i,'Age'])
    left += temp.loc[i,'%sum']

#inserting the text
txt = 0.0 #for left parameter in ax.text()

for i in temp.index:
    #for amount
    ax.text(temp.loc[i,'%sum']/2 + txt,0.15,f"{temp.loc[i,'sum_in_billions']}B",
        va = 'center', ha='center',fontsize=14, color='white')

    #for age grp
    ax.text(temp.loc[i,'%sum']/2 + txt,- 0.20 ,f"{temp.loc[i,'Age']}",
        va = 'center', ha='center',fontsize=12, color='white')

    txt += temp.loc[i,'%sum']
```

```

#removing the axis lines
for s in ['top','left','right','bottom']:
    ax.spines[s].set_visible(False)

#customizing ticks
ax.set_xticks([])
ax.set_yticks([])
ax.set_xlim(0,1)

#plot title
ax.set_title('Age Group Purchase Amount Distribution',{'font':'serif', 'size':
    ↳15,'weight':'bold'})

#Distribution of Purchase Amount
↳per Transaction

ax1 = fig.add_subplot(gs[1])

#plotting the visual
ax1.bar(temp['Age'],temp['per_purchase'],color = color_map,zorder = 2,width = 0.
    ↳3)

#adding average transaction line
avg = round(df['Purchase'].mean())

ax1.axhline(y = avg, color = 'red', zorder = 0,linestyle = '--')

#adding text for the line
ax1.text(0.4,avg + 300, f"Avg. Transaction Amount ${avg:.0f}",
    {'font':'serif','size' : 12},ha = 'center',va = 'center')

#adjusting the ylimits
ax1.set_ylim(0,11000)

#adding the value_counts
for i in temp.index:
    ax1.text(temp.loc[i,'Age'],temp.loc[i,'per_purchase']/2,f"${temp.
    ↳loc[i,'per_purchase']:.0f}",
        {'font':'serif','size' : 12,'color':'white','weight':'bold' },ha =
    ↳'center',va = 'center')

#adding grid lines
ax1.grid(color = 'black',linestyle = '--', axis = 'y', zorder = 0, dashes =
    ↳(5,10))

```

```

#removing the axis lines
for s in ['top','left','right']:
    ax1.spines[s].set_visible(False)

#adding axis label
ax1.set_ylabel('Purchase Amount',fontweight = 'bold',fontsize = 12)
ax1.set_xticklabels(temp['Age'],fontweight = 'bold',fontsize = 12)

#setting title for visual
ax1.set_title('Average Purchase Amount per Transaction',{'font':'serif', 'size':
    ↳15,'weight':'bold'})

# creating kdeplot for purchase amount
↳distribution

ax3 = fig.add_subplot(gs[2,:])

#plotting the kdeplot
sns.kdeplot(data = df, x = 'Purchase', hue = 'Age', palette = color_map,fill =
    ↳True, alpha = 0.5,
    ax = ax3)

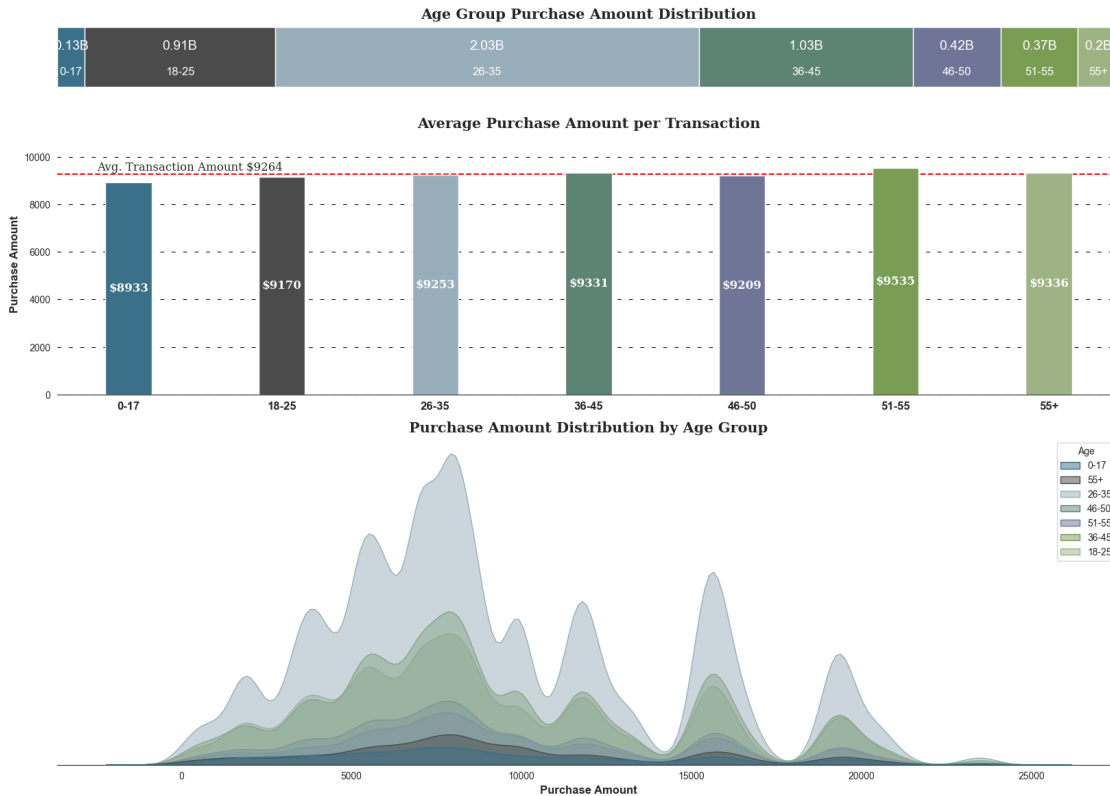
#removing the axis lines
for s in ['top','left','right']:
    ax3.spines[s].set_visible(False)

# adjusting axis labels
ax3.set_yticks([])
ax3.set_ylabel('')
ax3.set_xlabel('Purchase Amount',fontweight = 'bold',fontsize = 12)

#setting title for visual
ax3.set_title('Purchase Amount Distribution by Age Group',{'font':'serif',
    ↳'size':15,'weight':'bold'})

plt.show()

```



Insights 1. Total Sales Comparison

- Age group between 26 - 45 accounts to almost 60% of the total sales suggesting that Walmart's Black Friday sales are most popular among these age groups.
- The age group 0-17 has the lowest sales percentage (2.6%), which is expected as they may not have as much purchasing power. Understanding their preferences and providing special offers could be beneficial, especially considering the potential for building customer loyalty as they age.

2. Average Transaction Value

- While there is not a significant difference in per purchase spending among the age groups, the 51-55 age group has a relatively low sales percentage (7.2%) but they have the highest per purchase spending at 9535. Walmart could consider strategies to attract and retain this high-spending demographic.

3. Distribution of Purchase Amount

- As seen above, the purchase amount for all age groups is not normally distributed.

Confidence Interval Construction: Estimating Average Purchase Amount per Transaction

1. Step 1 - Building CLT Curve

- As seen above, the purchase amount distribution is not Normal. So we need to use Central Limit Theorem. It states the distribution of sample means will approximate a normal

distribution, regardless of the underlying population distribution

2. Step 2 - Building Confidence Interval

- After building CLT curve, we will create a confidence interval predicting population mean at 95% Confidence level. Note - We will use different sample sizes of [100,1000,5000,50000]

```
[132]: #defining a function for plotting the visual for given confidence interval
```

```
def plot(ci):

    #setting the plot style
    fig = plt.figure(figsize = (15,15))
    gs = fig.add_gridspec(4,1)

    #creating separate data frames

    df_1 = df.loc[df['Age'] == '0-17', 'Purchase']
    df_2 = df.loc[df['Age'] == '18-25', 'Purchase']
    df_3 = df.loc[df['Age'] == '26-35', 'Purchase']
    df_4 = df.loc[df['Age'] == '36-45', 'Purchase']
    df_5 = df.loc[df['Age'] == '46-50', 'Purchase']
    df_6 = df.loc[df['Age'] == '51-55', 'Purchase']
    df_7 = df.loc[df['Age'] == '55+', 'Purchase']

    #sample sizes and corresponding plot positions
    sample_sizes = [(100,0),(1000,1),(5000,2),(50000,3)]

    #number of samples to be taken from purchase amount
    bootstrap_samples = 20000

    samples1,samples2,samples3,samples4,samples5,samples6,samples7 = _
    ↪ {}, {}, {}, {}, {}, {}, {}

    for i,x in sample_sizes:
        l1,l2,l3,l4,l5,l6,l7 = [], [], [], [], [], [], []

        for j in range(bootstrap_samples):

            #creating random 5000 samples of i sample size
            bootstrapped_samples_1 = np.random.choice(df_1,size = i)
            bootstrapped_samples_2 = np.random.choice(df_2,size = i)
            bootstrapped_samples_3 = np.random.choice(df_3,size = i)
            bootstrapped_samples_4 = np.random.choice(df_4,size = i)
            bootstrapped_samples_5 = np.random.choice(df_5,size = i)
            bootstrapped_samples_6 = np.random.choice(df_6,size = i)
            bootstrapped_samples_7 = np.random.choice(df_7,size = i)
```

```

#calculating mean of those samples
sample_mean_1 = np.mean(bootstrapped_samples_1)
sample_mean_2 = np.mean(bootstrapped_samples_2)
sample_mean_3 = np.mean(bootstrapped_samples_3)
sample_mean_4 = np.mean(bootstrapped_samples_4)
sample_mean_5 = np.mean(bootstrapped_samples_5)
sample_mean_6 = np.mean(bootstrapped_samples_6)
sample_mean_7 = np.mean(bootstrapped_samples_7)

#appending the mean to the list
l1.append(sample_mean_1)
l2.append(sample_mean_2)
l3.append(sample_mean_3)
l4.append(sample_mean_4)
l5.append(sample_mean_5)
l6.append(sample_mean_6)
l7.append(sample_mean_7)

#storing the above sample generated
samples1[f'{ci}%_{i}'] = l1
samples2[f'{ci}%_{i}'] = l2
samples3[f'{ci}%_{i}'] = l3
samples4[f'{ci}%_{i}'] = l4
samples5[f'{ci}%_{i}'] = l5
samples6[f'{ci}%_{i}'] = l6
samples7[f'{ci}%_{i}'] = l7

#creating a temporary dataframe for creating kdeplot
temp_df = pd.DataFrame(data = {'0-17':l1,'18-25':l2,'26-35':l3,'36-45':
↪l4,'46-50':l5,'51-55':l6,'55+':l7})

                                                                 #plotting kdeplots

#plot position
ax = fig.add_subplot(gs[x])

#plots
for p,q in [('#3A7089', '0-17'),('#4b4b4c', '18-25'),('#99AE8B', '
↪26-35'),('#5C8374', '36-45'),('#6F7597', '46-50'),
            ('#7A9D54', '51-55'),('#9EB384', '55+')]:

    sns.kdeplot(data = temp_df,x = q,color =p ,fill = True, alpha = 0.
↪5,ax = ax,label = q)

#removing the axis lines

```

```

    for s in ['top','left','right']:
        ax.spines[s].set_visible(False)

    # adjusting axis labels
    ax.set_yticks([])
    ax.set_ylabel('')
    ax.set_xlabel('')

    #setting title for visual
    ax.set_title(f'CLT Curve for Sample Size = {i}', {'font':'serif', 'size':
↪11, 'weight':'bold'})

    plt.legend()

    #setting title for visual
    fig.suptitle(f'{ci}% Confidence Interval', font = 'serif', size = 18, weight_
↪= 'bold')

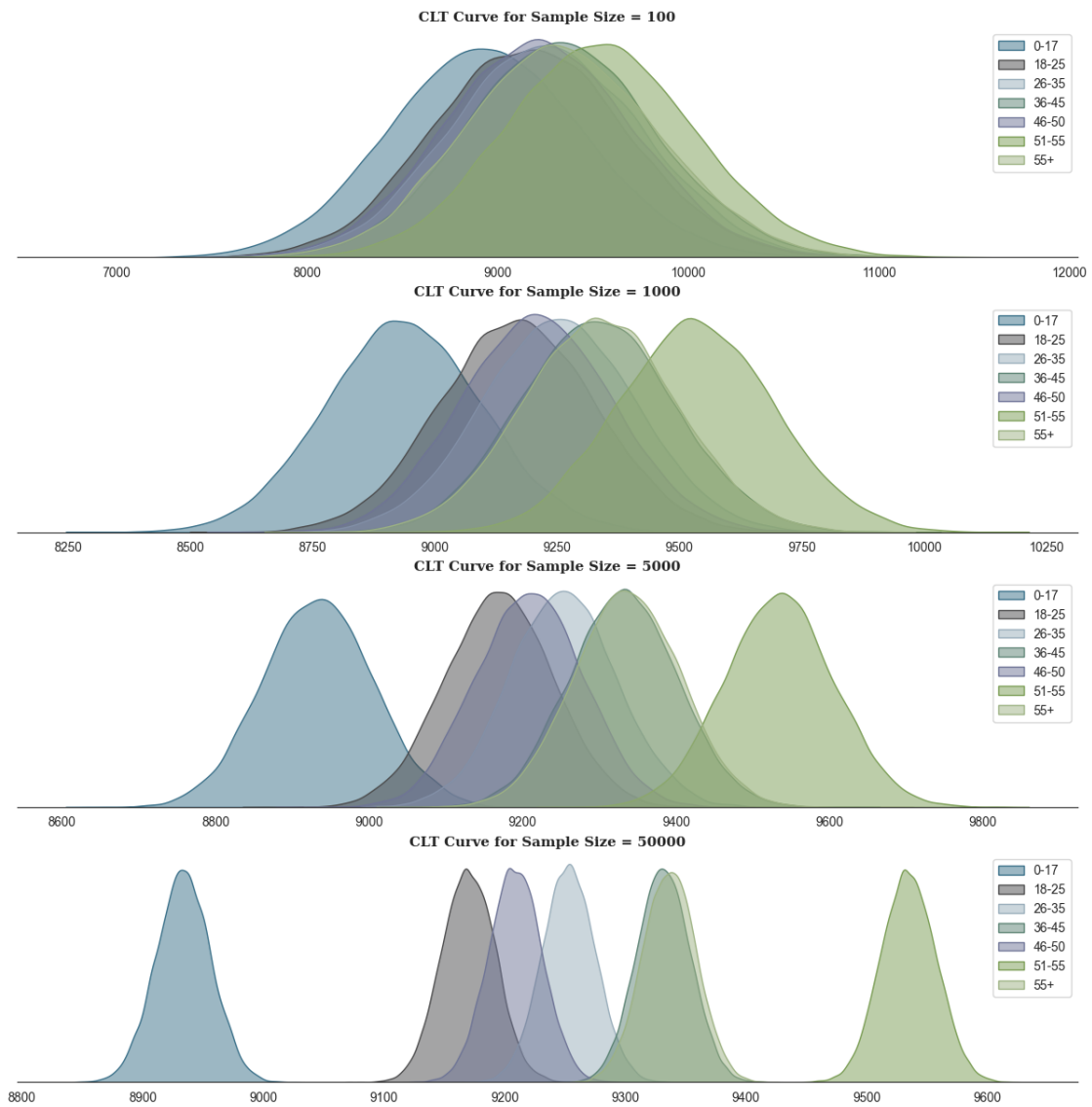
    plt.show()

    return samples1,samples2,samples3,samples4,samples5,samples6,samples7

```

```
[133]: samples1,samples2,samples3,samples4,samples5,samples6,samples7 = plot(95)
```

95% Confidence Interval



Are confidence intervals of customer's age-group spending overlapping?

```
[134]: #setting the plot style
fig,ax = plt.subplots(figsize = (20,5))

#list for collecting ci for given cl
ci_1,ci_2,ci_3,ci_4,ci_5,ci_6,ci_7 =_
↳ ['0-17'], ['18-25'], ['26-35'], ['36-45'], ['46-50'], ['51-55'], ['55+']
```

```

#finding ci for each sample size
#samples = [samples1,samples2,samples3,samples4,samples5,samples6,samples7]

samples = 
    ↪[(samples1,ci_1),(samples2,ci_2),(samples3,ci_3),(samples4,ci_4),(samples5,ci_5),(samples6

for s,c in samples:
    for i in s:
        s_range = confidence_interval(s[i],95)
        c.append(f"CI = ${s_range[0]:.0f} - ${s_range[1]:.0f}, Range = 
    ↪{(s_range[1] - s_range[0]):.0f}")

#plotting the summary

#contents of the table
ci_info = [ci_1,ci_2,ci_3,ci_4,ci_5,ci_6,ci_7]

#plotting the table
table = ax.table(cellText = ci_info, cellLoc='center',
                collabels = ['Age Group','Sample Size = 100','Sample Size = 
    ↪1000','Sample Size = 5000','Sample Size = 50000'],
                colLoc = 'center',colWidths = [0.1,0.225,0.225,0.225,0.225],bbox=
    ↪=[0, 0, 1, 1])

table.set_fontsize(13)

#removing axis
ax.axis('off')

#setting title
ax.set_title(f"95% Confidence Interval Summary",{ 'font':'serif', 'size':
    ↪14, 'weight':'bold'})

plt.show()

```

95% Confidence Interval Summary				
Age Group	Sample Size = 100	Sample Size = 1000	Sample Size = 5000	Sample Size = 50000
0-17	CI = 7954 – 9946, Range = 1992	CI = 8620 – 9256, Range = 637	CI = 8791 – 9078, Range = 287	CI = 8888 – 8978, Range = 90
18-25	CI = 8202 – 10168, Range = 1966	CI = 8856 – 9484, Range = 628	CI = 9029 – 9309, Range = 280	CI = 9126 – 9213, Range = 87
26-35	CI = 8276 – 10237, Range = 1961	CI = 8940 – 9564, Range = 625	CI = 9112 – 9393, Range = 281	CI = 9209 – 9297, Range = 88
36-45	CI = 8368 – 10328, Range = 1960	CI = 9027 – 9648, Range = 621	CI = 9193 – 9471, Range = 278	CI = 9288 – 9375, Range = 87
46-50	CI = 8261 – 10181, Range = 1921	CI = 8906 – 9520, Range = 613	CI = 9072 – 9346, Range = 274	CI = 9165 – 9252, Range = 87
51-55	CI = 8566 – 10544, Range = 1979	CI = 9221 – 9854, Range = 634	CI = 9393 – 9678, Range = 285	CI = 9490 – 9579, Range = 89
55+	CI = 8361 – 10335, Range = 1974	CI = 9024 – 9650, Range = 626	CI = 9198 – 9474, Range = 276	CI = 9292 – 9380, Range = 88

Insights

1. Sample Size

- The analysis highlights the importance of sample size in estimating population parameters. It suggests that as the sample size increases, the confidence intervals become narrower and more precise. In business, this implies that larger sample sizes can provide more reliable insights and estimates.
- 2. Confidence Intervals and customer spending patterns
- From the above analysis, we can see that the confidence interval overlap for some of the age groups. We can club the average spending into following age groups - 0 - 17 - Customers in this age group have the lowest spending per transaction 18 - 25, 26 - 35, 46 - 50 - Customers in these age groups have overlapping confidence intervals indicating similar buying characteristics 36 - 45, 55+ - Customers in these age groups have overlapping confidence intervals indicating and similar spending patterns 51 - 55 - Customers in this age group have the highest spending per transaction

3. Population Average

- We are 95% confident that the true population average for following age groups falls between the below range -

0 - 17 = \$ 8,888 to 8,979 18 - 25 = \$ 9,125 to 9,213 26 - 35 = \$ 9,209 to 9,297 36 - 45 = \$ 9,288 to 9,376 46 - 50 = \$ 9,165 to 9,253 51 - 55 = \$ 9,490 to 9,579 55+ = \$ 9,292 to 9,381

4. How can Walmart leverage this conclusion to make changes or improvements?

4.1. Targeted Marketing

- Knowing that customers in the 0 - 17 age group have the lowest spending per transaction, Walmart can try to increase their spending per transaction by offering them more attractive discounts, coupons, or rewards programs. Walmart can also tailor their product selection and marketing strategies to appeal to the preferences and needs of this age group

4.2. Customer Segmentation

- Since customers in the 18 - 25, 26 - 35, and 46 - 50 age groups exhibit similar buying characteristics, and so do the customers in 36 - 45 and 55+, Walmart can optimize its product selection to cater to the preferences of these age groups. Also, Walmart can use this information to adjust their pricing strategies for different age groups.

4.3 Premium Services

- Recognizing that customers in the 51 - 55 age group have the highest spending per transaction, Walmart can explore opportunities to enhance the shopping experience for this demographic. This might involve offering premium services, personalized recommendations, or loyalty programs that cater to the preferences and spending habits of this age group.

Recommendations

1.Target Male Shoppers

- Since male customers account for a significant portion of Black Friday sales and tend to spend more per transaction on average, Walmart should tailor its marketing strategies and product

offerings to incentivize higher spending among male customers while ensuring competitive pricing for female-oriented products.

2. Focus on 26 - 45 Age Group

- With the age group between 26 and 45 contributing to the majority of sales, Walmart should specifically cater to the preferences and needs of this demographic. This could include offering exclusive deals on products that are popular among this age group.

3. Engage Younger Shoppers

- Knowing that customers in the 0 - 17 age group have the lowest spending per transaction, Walmart can try to increase their spending per transaction by offering them more attractive discounts, coupons, or rewards programs. It's essential to start building brand loyalty among younger consumers.

4. Customer Segmentation

- Since customers in the 18 - 25, 26 - 35, and 46 - 50 age groups exhibit similar buying characteristics, and so do the customers in 36 - 45 and 55+, Walmart can optimize its product selection to cater to the preferences of these age groups. Also, Walmart can use this information to adjust their pricing strategies for different age groups.

5. Enhance the 51 - 55 Age Group Shopping Experience

- Considering that customers aged 51 - 55 have the highest spending per transaction, Walmart offer them exclusive pre-sale access, special discount or provide personalized product recommendations for this age group. Walmart can also introduce loyalty programs specifically designed to reward and retain customers in the 51 - 55 age group.

6. Post-Black Friday Engagement

- After Black Friday, walmart should engage with customers who made purchases by sending follow-up emails or offers for related products. This can help increase customer retention and encourage repeat business throughout the holiday season and beyond.